Final presentation: 15-minute script

Four parts, one per speaker, in this order: Prakhar opens with the problem, Krish explains how we decided and shows the proof, Aryan runs the live demo, Ved closes. The deck is ordered the same way, so nobody skips slides. The script runs about 14:30, leaving 30 seconds of buffer. The slides are presentation/AddaWale-Finals-Presentation.pdf (16 slides; printable copies of this script and the demo script are in the same folder); the app runs at http://127.0.0.1:3000.

Starts	Part and speaker	Slides / screen
0:00	Part 1. The problem, Prakhar (3:15)	Slides 1-3, then the live Overview and Work queue
3:15	Part 2. How we decided, and the proof, Krish (5:00)	Slides 4-9
8:15	Part 3. Live demo, Aryan (5:15)	Live app: customer page, Call Mode, blocked customer, supervisor review
13:30	Part 4. Close, Ved (1:00)	Slides 10-11

Slides 12-16 are marked *Backup slide* and are not presented: the live demo shows the same screens. Use them only if the app fails (see the end).

Speak at a calm pace. Words in bold in the "Do" column are buttons or screen names. Each part ends with a hand-off line naming the next speaker.

Before you start

1. Backend with a fresh review store and Gemini off: COPILOT_STATE=data/finals-demo-reviews.sqlite3 COPILOT_GEMINI_ENABLED=0 .venv/bin/python -m uvicorn app.main:app --host 127.0.0.1 --port 8000. Frontend as a production build: cd frontend && npm run build && npm run start.

2. Sign in as supervisor / supervisor-demo. Open every page once so it is warm, then reload /call/JEV-1240 so the call starts empty.

3. Two windows, switched with Alt+Tab: the deck PDF in full screen, and the browser on Overview.

4. Keep the call note in a text file ready to paste: Customer lost his job last month. He can pay $315.86 on October 15. Asked us to call back next week.

5. Rehearse the Overview scroll: What's at stake shows one big number at a time and changes as you scroll ($69.8M, then $18.0M, then $338.6K). Scroll slowly.

Part 1: The problem (Prakhar, 3:15)

Time	Do	Say
0:00	Slide 1 (cover)	"Good morning. We're Adda Wale: Krish, Aryan, Ved and I'm Prakhar. We built Collections Copilot: it tells a bank's collections agent what to do with each overdue customer, in the right order, within policy, with the reason shown."
0:20	Slide 2	"Collections means calling customers who already owe the bank money. It isn't sales. Before every call, an agent has to check eight things: restrictions like bankruptcy, whether a lawyer represents the customer, disputes, vulnerability, hardship, an existing promise to pay, when contact is legal, and only then the payment request. The facts sit in different systems and the rules in 19 policy documents. Miss one, and it's a breach or a harmed customer."
1:00	Slide 3	"So we ran those checks across all 96,000 open cases in the data. 760 cases were contacted more often than the rules allow this week. 1,040 contacts happened outside legal hours or on a holiday. And 18,758 customers in financial hardship have no support plan yet. Each of these numbers is one SQL query, and you can see it in the app. Let me show you."
1:35	Alt+Tab to the browser, Overview. Open Definition and SQL on *Cases contacted more often than allowed*	"This is the manager's view. Every tile opens its definition and the exact query, so nobody has to trust our numbers."
1:55	Scroll slowly through What's at stake; pause on each number	"And the money behind it: $69.8 million exposed on cases about to be written off; $18 million owed by customers in hardship with no plan; $338 thousand spent on contacts in the last 30 days."
2:30	Scroll to Today's first step	"Every night we plan every open case. Fewer than half should simply be asked to pay; the rest need something else first: a reminder, a hardship offer, or no contact at all."
2:50	Click Work queue	"And this is one agent's day: 296 cases in a fixed order, not a black-box score. Each row says why it's there and when the agent may legally call next."
3:05	Alt+Tab back to the slides, slide 4	"Before we show the product, Krish will explain how we decided what to build, and why."

Part 2: How we decided, and the proof (Krish, 5:00)

Time	Do	Say
3:15	Slide 4	"Before writing code we set five constraints. Compliance first: a wrong step is a breach, not a lower score. Explainable: every recommendation names its record and policy section. Customer text stays inside the bank by default. It must work on the next data release, not just this one. And every number is measured, not claimed."
3:45	Slide 4, point at the diagram	"Then one rule picked the technology for each job. If the answer is written in policy, we use rules: contact hours, the weekly cap, restrictions, eligibility. If it's a pattern in free text and we have labelled examples, a small model: hardship cues in agent notes. If it's language whose output we can check against a source, an LLM proposes and the app verifies. Otherwise, a person decides. So the LLM goes where language helps, and it never touches numbers or compliance steps."
4:25	Slide 5	"This is the backend. The browser has no business logic. FastAPI checks the user's role on every request, not by hiding buttons, and calls one of three engines. The rules engine turns each policy step into code; the brief, the live call, the queue and the nightly batch all use the same function. The Ask engine turns a question into reviewed SQL with bound parameters, and searches policy with BM25, because answers must cite an exact section and version."
5:00	Slide 5, point at DuckDB and SQLite	"Customer data lives in DuckDB, a columnar database in one file: 1 million customers, 2.7 million contacts, 760 thousand notes, and a 30-day scan takes milliseconds. The app opens it read-only. The only writes are reviews and the audit log, in a separate SQLite file that is hash-chained, so tampering shows. Gemini is optional and sits behind the API."
5:25	Slide 6	"Now the question every judge should ask: why not just use an LLM to find hardship? We didn't assume; we measured. Same 126 held-out notes, 38 with hardship. Our detector found all 38 with zero false alarms. Gemini also found 38, but raised four false alarms."
5:50	Slide 6, point at the line under the table	"Three of those four were notes tagged 'customer vulnerable, financial hardship'. In this bank's notes, that's a vulnerability tag, not hardship, and our policy treats it differently. Gemini also took 1.4 seconds a note instead of under a millisecond, would send every note outside the bank, and gives no reason. For all 760 thousand notes, that's 12 days of calls."
6:15	Slide 6	"So the small model flags hardship, and the LLM only proposes where a person checks. We chose Gemini 3.5 Flash-Lite because it's fast, returns structured JSON we validate, and the larger model was overloaded the day we tested."
6:30	Slide 7	"The same rules plan the whole book every night: 93,320 customers in 21 seconds on one laptop, with zero errors. That's the same function behind every brief and every call you'll see in the demo."
7:00	Slide 8	"And it isn't tied to this dataset. The snapshot date, the holidays, the weekly cap per case, the policy versions and the programme rules are all read from the data. We built a fake March 2027 release with a stricter cap and new policy versions; the plan followed it with no code changes."

Time	Do	Say
7:35	Slide 9	"We also measured our own detector honestly: recall went from 68 to 95 percent and precision from 90 to 98, and on notes we never looked at, it found 36 of 38 with no false alarms. The caveat is on the slide: this synthetic data reuses phrasings, so it's coverage of this dataset, not real-world accuracy."
8:05	Stay on slide 9	"Now Aryan will show you the product live."

Part 3: Live demo (Aryan, 5:15)

Time	Do	Say
8:15	Alt+Tab to the browser. Press Ctrl+K, type SAV-6847, open the customer. History & contact tab	"Ctrl+K takes us to any customer. This is one view of a customer across every source system."
8:30	Point at the calendar	"When can we call her? Monday and Tuesday are blocked: three attempts already in seven days. Wednesday is the National Day for Truth and Reconciliation. So Thursday, 7 am to 9 pm in her own time zone. The holidays are computed for any year, not typed in."
9:00	Ctrl+K, type JEV, choose Call: Jérôme Vallée	"Now a live call. Jérôme has an auto loan, 86 days past due, and prefers French; the script tells the agent to offer French."
9:15	Who answered? Customer. Tick Recording notice and Verify identity	"Nothing about the debt is shown until the recording notice is read and identity is verified."
9:35	Tick Hardship, choose Job loss. Point at the live plan and Hardship options	"He says he lost his job. The plan changes immediately: offer a hardship programme before any payment request. These are his real numbers: $631.72 a month drops to $315.86 for three months, which the agent can approve. A term extension is $578.71 but needs a specialist. Skipping a payment isn't offered, and it says why: 86 days is over the 60-day limit."
10:20	Choose on *3-month reduced payment*	"He takes the reduced payment."
10:30	Income 2400, expenses 1900. Promise 315.86 on 15 Oct	"Can he afford a promise? $315.86 is 63 percent of what he has left each month, so the app warns the agent. It's within 21 days, so the date is accepted."
11:00	Paste the call note into Call notes, press Extract from notes	"Agents type fast. Extract turns notes into proposals: job loss, the promise, a callback. Each shows the exact sentence it came from, and nothing changes until the agent accepts it."
11:30	Go to wrap-up	"The call note is written for the agent, with the policy cited. The outcome codes are set, and one reminder is scheduled before the 15th; it counts toward the weekly cap."
11:55	Submit for supervisor review	"It goes to a supervisor. The app never contacts the customer itself."
12:05	Ctrl+K, HAC-4447, open Call	"And when an agent must not call: this customer filed a consumer proposal. The call page is blocked, the controls are off, and contact goes through the trustee."
12:25	Click the citation POL-COLL-001 v3.0 §7.3	"Every step cites its policy, and every citation opens the exact section: here, all collection contact stops once a consumer proposal is filed."
12:40	Open Reviews, open the call note, Approve with a reason such as "Programme within agent authority"	"The supervisor sees the note and the outcome codes, and approves, edits or rejects with a reason. Every action goes into the audit log."
13:15	Alt+Tab back to the slides, slide 10	"Ved will take us to the close."

If time is short, cut in this order: the affordability step (10:30), then the Extract step (11:00). Never cut the blocked customer: it is the compliance proof.

Don't click policy citations during the JEV-1240 call before Submit for supervisor review: the app asks "Leave this call?" because the recorded outcomes would be lost. If that prompt appears, choose Cancel to stay. Ctrl+K, the menu and the citation on the blocked HAC-4447 call are not affected.

Part 4: Close (Ved, 1:00)

Ved reads from this page. No live clicking.

Time	Do	Say
13:30	Slide 10	"Managers can also ask questions in plain English. Every answer shows its SQL. If a question can't be answered correctly, it asks for clarification instead of guessing, and it refuses requests like exporting personal data. It matched all 12 of the organisers' reference answers."
13:55	Slide 11	"To sum up: rules decide, AI suggests, and people approve. 559 tests pass, and we checked every screen in a real browser. We state our limits openly: the data is synthetic, and the app never contacts a customer; it only prepares and records."
14:20	Stay on slide 11	"Thank you. We're happy to take your questions."

Questions: who answers what

The person named first answers; the others add only if needed. Full answers are in docs/finals-judge-qa.md.

First	Backup	Topic
Krish	Prakhar	Architecture, model and LLM choice, scale, new data
Aryan	Prakhar	The screens, Call Mode, the redesign, Customer 360 and identity matching
Prakhar	Krish	The numbers on the Overview, Ask Collections, data quality
Ved	Krish	"Does the app contact customers?" and "Is this real data?"

Ved's two answers, word for word:

- *Does it contact customers?* "No. It never sends a message, places a call or changes a record. It prepares the plan and a supervisor approves it."

- *Is this real customer data?* "No. It's the organisers' synthetic Maple Bank data. No real customer data is used."

If something breaks

- App down or slow: Aryan says "Let me show you the same flow on the slides" and presents backup slides 13 (SAV-6847 calendar), 14 (JEV-1240 hardship options) and 15 (Call Mode) with the same lines. Do not apologise; keep the same narration. Slides 12 (the brief) and 16 (work queue) are extra backups.

- Running long: cut the affordability step, then the Extract step, then slide 9.

- Someone freezes: the next speaker picks up from the hand-off line; every part stands on its own.